

Tourist Recommender System - Database Documentation

Overview

This database supports a tourist recommendation system that uses:

- **Module I:** User profile analysis using Natural Language Classification
 - **Module II:** POI popularity analysis from social media (Twitter)
 - **Module III:** K-means clustering to group POIs by geographical location
 - **Module IV:** Genetic Algorithm to solve Time-Dependent Team Orienteering Problem with Time Windows (TTDP/OPTW)
-

Database Structure

1. USER MANAGEMENT (3 tables)

users - Core user information

- Stores login credentials and basic profile
- Primary key: user_id

user_preferences - User's explicit preferences

- Categories like museums, parks, restaurants
- Weight values (0.0 to 1.0) indicate preference strength
- Links to: users

user_interests - Detected user interests

- Source can be: manual entry, survey, NLC (Natural Language Classification), or behavior analysis
 - Confidence scores from Watson NLC
 - Links to: users
-

2. POINTS OF INTEREST - POI (5 tables)

pois - Main POI information

- Uses Google Place ID as primary key
- Contains location (latitude/longitude), rating, popularity
- Expected visit duration for itinerary planning

poi_categories - POI type classifications

- Maps Google types to custom categories

- One POI can have multiple categories
- Links to: pois

poi_opening_hours - Operating hours

- Critical for TTDP constraint checking
- Stores hours per day of week (0=Sunday to 6=Saturday)
- Links to: pois

poi_time_windows - Multiple opening periods

- For POIs that close during lunch or have split hours
- Links to: poi_opening_hours

poi_photos - POI images

- Stores Google Places photo references and URLs
 - Links to: pois
-

3. TRAVEL INFORMATION (2 tables)

destinations - Tourist cities

- Cities where trips can be planned
- Contains timezone for time calculations

travel_times - Pre-calculated distance matrix

- Travel time and distance between every POI pair
 - Essential input for Genetic Algorithm (Module IV)
 - Supports multiple travel modes: walking, driving, transit
 - Links to: pois (origin and destination)
-

4. TRIP PLANNING (1 table)

trips - User's travel plans

- Date range and number of days
 - Daily time constraints (start time, end time, lunch break)
 - Accommodation location (hotel) as starting point
 - Links to: users, destinations
-

5. K-MEANS CLUSTERING - MODULE III (2 tables)

clusters - Daily POI groupings

- One cluster per day of the trip
- Contains centroid coordinates
- Groups nearby POIs to minimize travel time
- Links to: trips

cluster_pois - POIs assigned to clusters

- Maps which POIs belong to each day's cluster
 - Distance from POI to cluster centroid
 - Links to: clusters, pois
-

6. GENETIC ALGORITHM & ITINERARIES - MODULE IV (4 tables)

itineraries - Optimized daily routes

- GA-generated solutions (chromosomes)
- Fitness score measures solution quality
- Tracks penalties like lunch time invasion
- is_best_solution marks optimal route
- Links to: trips, clusters

itinerary_pois - Visit sequence

- Ordered list of POIs to visit (genes in chromosome)
- Visit order determines route
- Arrival/departure times calculated by GA
- Penalties if arriving outside opening hours
- Links to: itineraries, pois

itinerary_restaurants - Lunch suggestions

- Restaurants near user during lunch hours
- Links to: itineraries, pois

ga_executions - Algorithm performance logs

- Records GA parameters: population size, generations
- Tracks execution time and best fitness

- Useful for research analysis
 - Links to: trips
-

7. EVALUATION & FEEDBACK (2 tables)

poi_user_ratings - POI ratings by users

- 1-5 star ratings after visit
- Tracks if user actually visited the POI
- Used to improve recommendations
- Links to: users, pois, trips

itinerary_feedback - Survey responses

- Based on research paper's evaluation (Table 4)
 - 16 questions (P1-P16) covering:
 - P1-P2: How well system understands user
 - P3-P10: Accuracy of recommendations
 - P11-P14: Novelty (discovering new places)
 - P15-P16: User attitude towards system
 - All ratings on 1-5 scale
 - Links to: users, itineraries
-

8. SYSTEM PERFORMANCE (1 table)

api_cache - API response caching

- Reduces calls to Google Places API
 - Stores JSON responses
 - TTL (time to live) with expires_at
 - Improves performance and reduces costs
-

Key Relationships

User Flow

User → User Preferences/Interests → Filtered POIs → Trip Planning

Clustering Flow (Module III)

Trip + POIs → K-means Algorithm → Clusters (one per day) → Cluster POIs

Optimization Flow (Module IV)

Cluster + Travel Times → Genetic Algorithm → Itineraries → Itinerary POIs (ordered)

Feedback Loop

Itinerary → User Visits POIs → Ratings & Feedback → Improve Future Recommendations

Important Design Decisions

1. Why separate cluster_pois and itinerary_pois?

- **cluster_pois**: Shows which POIs are geographically grouped (Module III output)
- **itinerary_pois**: Shows the optimized visit order within a cluster (Module IV output)

2. Why store travel_times instead of calculating on-demand?

- Pre-calculation speeds up GA fitness function evaluation
- Genetic algorithm runs thousands of fitness evaluations
- Real-time API calls would be too slow and expensive

3. Why track GA execution parameters?

- Research project needs performance metrics
- Helps compare different GA configurations
- Important for final year project documentation

4. Time-dependent constraints (TTDP)

- Opening hours are strictly enforced
- Lunch time is protected (penalty if invaded)
- Daily time windows limit planning (e.g., 9 AM - 6 PM)

Critical Tables for Each Module

Module I (User Profiling):

- users, user_preferences, user_interests

Module II (Popularity Analysis):

- pois (popularity_score field)

Module III (K-means Clustering):

- clusters, cluster_pois

Module IV (Genetic Algorithm):

- itineraries, itinerary_pois, travel_times, poi_opening_hours

Evaluation:

- poi_user_ratings, itinerary_feedback
-

Notes for Implementation

Indexes

- All foreign keys are indexed automatically
- Location-based queries use indexes on latitude/longitude
- Search queries indexed on commonly filtered fields

Views

Three views are provided for common queries:

1. **v_itinerary_details**: Complete daily itinerary with POI info
2. **v_trip_summary**: Trip overview with destination details
3. **v_poi_popularity**: Combined popularity metrics from multiple sources

Constraints

- Rating fields limited to 1-5 range using CHECK constraints
 - Unique constraints prevent duplicate travel_times entries
 - Foreign keys with CASCADE delete maintain referential integrity
-

Database Size Estimates

For a city with 500 POIs:

- **pois**: 500 rows
- **travel_times**: ~250,000 rows (500 × 500 combinations)
- **poi_opening_hours**: ~3,500 rows (500 POIs × 7 days)
- Per trip with 3 days:
 - **clusters**: 3 rows
 - **cluster_pois**: ~30-50 rows (10-15 POIs per day)
 - **itineraries**: 3-30 rows (depending on how many solutions to save)

- **itinerary_pois:** ~30-150 rows

Total storage: Moderate size, easily manageable with standard MySQL